Ficha rpcw2026-ficha-medicina

23 de Março de 2026, 9h, Ed.2 - sala 0.14

Mestrado em Engenharia Informática (1º ano)

Sinopsis

O objectivo principal desta ficha é avaliar os conhecimentos obtidos durante as aulas na especificação e desenvolvimento de ontologias e de aplicações que tiram partido destas.

Antes de começares, lê atentamente até ao fim para ficares com uma percepção do todo que se pretende. Vais ver que tomarás decisões mais acertadas depois de uma leitura completa.

Os resultados finais deverão ser enviados ao docente da seguinte forma:

- Criar uma pasta no GitHub para a ficha:
  - O repositório no GitHub deverá chamar-se `RPCW2026-Ficha-Medicina`;
  - Dentro do repositório deverá haver um ficheiro, `PR.md`, contendo uma descrição do que fez e como fez, contendo instruções se necessário, para o docente poder replicar os cenários na correção.

Query `CONSTRUCT`

Uma query SPARQL `CONSTRUCT` é um tipo de consulta que, em vez de retornar uma tabela de dados (como o `SELECT`), retorna um novo grafo RDF, ou seja, retorna uma lista de triplos `(sujeito predicado objeto)`.

Ela é utilizada para transformar dados existentes, criar novas relações, novos triplos, ou exportar subconjuntos de um grafo num formato semântico.

Como Funciona

A query `CONSTRUCT` baseia-se num modelo (template). Se as condições no bloco `WHERE` forem verificadas, o SPARQL preenche o template com os valores encontrados e gera triplos válidos `(Sujeito --> Predicado --> Objeto)`.

Estrutura Básica

```
CONSTRUCT {?sujeito ?predicado ?objeto . }
WHERE {?sujeito ?p ?o . }
```

Exemplo: Biblioteca temporal

Considerando a ontologia da biblioteca trabalhada nas últimas aulas. Apresentam-se a seguir algumas situações resolvidas com queries `CONSTRUCT`.

Experimenta-as no GraphDB e verifica a criação dos novos triplos.

1. Criar uma relação direta entre Bibliotecário e Livro

Atualmente, a ontologia diz que um `:Bibliotecario` trabalha numa `:Biblioteca` e um `:Livro está numa :Biblioteca`. Podemos usar uma query `CONSTRUCT` para criar uma relação direta (ex: :gereLivro) entre o funcionário e os livros que estão sob a sua responsabilidade.

```
PREFIX : <http://example.org/biblioteca-temporal#>

CONSTRUCT {
  ?bibliotecario :gereLivro ?livro .
}
WHERE {
  ?bibliotecario a :Bibliotecario ;
                 :trabalhaEm ?biblioteca .
  ?livro :estaEm ?biblioteca .
}
```

2. Identificar Co-Autores

Se dois autores escreveram o mesmo livro, podemos gerar um novo triplo que indique explicitamente que eles são colaboradores (`:colaborouCom`).

```
PREFIX : <http://example.org/biblioteca-temporal#>

CONSTRUCT {
  ?autor1 :colaborouCom ?autor2 .
}
WHERE {
  ?livro :temAutor ?autor1 , ?autor2 .
  FILTER(?autor1 != ?autor2)
}
```

3. Mapear para um Vocabulário Externo (FOAF)

Muitas vezes queremos que a nossa ontologia seja compatível com padrões comunitários. Este exemplo transforma as instâncias de `:Pessoa` (ou subclasses como `:Autor`) em instâncias de `foaf:Person`, mapeando também o nome (a ontologia `foaf`, "Friend of a friend" tornou-se famosa quando o facebook começou a usá-la para representar as relações de amizade entre membros da comunidade).

```
PREFIX : <http://example.org/biblioteca-temporal#>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

CONSTRUCT {
  ?pessoa a foaf:Person ;
```

```
            foaf:name ?nome .
  }
  WHERE {
    ?pessoa a :Pessoa .
    # Supondo que o nome está no rdfs:label ou numa propriedade de nome
    OPTIONAL { ?pessoa rdfs:label ?nome }
  }
```

Inserção de novos triplos

Se precisares de inserir novos triplos na ontologia, novos indivíduos, novas relações ou propriedades ou novas classes, podes fazê-lo programaticamente através de uma script python ou através de uma query SPARQL `INSERT`.

As queries `INSERT` têm a seguinte forma:

```
INSERT DATA {
    s1 p1 o1 .
    s2 p2 o2 .
    ...
}
```

em que s1, p1, o1, ..., identificam indivíduos, propriedades ou valores.

Exemplo, considerando a biblioteca:

```
INSERT DATA {
    :gereLivro a owl:ObjectProperty ;
        rdfs:domain :Bibliotecario ;
        rdfs:range :Livro .
    :Zeno :gereLivro :Livro_Temporal_040, :Livro_Temporal_042,
            :Livro_Temporal_036, :Livro_Temporal_045 .
    ...
}
```

Concluindo

Usando uma sequência de queries `CONSTRUCT` e `INSERT` podes expandir uma ontologia mediante as condições que entenderes.

Exercício: Era uma vez... uma ontologia médica...

```
@prefix : <http://www.example.org/disease-ontology#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
```

```
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix swrl: <http://www.w3.org/2003/11/swrl#> .
@prefix swrlb: <http://www.w3.org/2003/11/swrlb#> .

:Ontology a owl:Ontology .

# Classes
:Disease a owl:Class .
:Symptom a owl:Class .
:Treatment a owl:Class .
:Patient a owl:Class .

# Properties
:hasSymptom a owl:ObjectProperty ;
    rdfs:domain :Disease ;
    rdfs:range :Symptom .

:hasTreatment a owl:ObjectProperty ;
    rdfs:domain :Disease ;
    rdfs:range :Treatment .

:exhibitsSymptom a owl:ObjectProperty ;
    rdfs:domain :Patient ;
    rdfs:range :Symptom .

:hasDisease a owl:ObjectProperty ;
    rdfs:domain :Patient ;
    rdfs:range :Disease .

:receivesTreatment a owl:ObjectProperty ;
    rdfs:domain :Patient ;
    rdfs:range :Treatment .

# Disease instances
:Flu a :Disease ;
    :hasSymptom :Fever, :Cough, :SoreThroat ;
    :hasTreatment :Rest, :Hydration, :AntiviralDrugs .

:Diabetes a :Disease ;
    :hasSymptom :IncreasedThirst, :FrequentUrination, :Fatigue ;
    :hasTreatment :InsulinTherapy, :DietModification, :Exercise .

# Symptom instances
:Fever a :Symptom .
:Cough a :Symptom .
:SoreThroat a :Symptom .
:IncreasedThirst a :Symptom .
:FrequentUrination a :Symptom .
:Fatigue a :Symptom .

# Treatment instances
:Rest a :Treatment .
:Hydration a :Treatment .
```

```
:AntiviralDrugs a :Treatment .
:InsulinTherapy a :Treatment .
:DietModification a :Treatment .
:Exercise a :Treatment .


# Patient instances
:Patient1 a :Patient ;
    :name "Paul Harrods"
    :exhibitsSymptom :Fever ;
    :exhibitsSymptom :Cough ;
    :exhibitsSymptom :SoreThroat .

:Patient2 a :Patient ;
    :name "Ana Montana"
    :exhibitsSymptom :IncreasedThirst ;
    :exhibitsSymptom :FrequentUrination ;
    :exhibitsSymptom :Fatigue .
```

Povoamento

Analisa a ontologia médica acima (está disponível no ficheiro `medical.ttl`) e sobre ela realiza as seguintes operações:

Junto com este enunciado foram disponibilizados 3 datasets em CSV e um em JSON:

- `Disease_Syntoms.csv` que contem uma lista de doenças e a cada uma associa uma lista de sintomas;
- `Disease_Description.csv` que contem uma lista de pares: doença e respetiva descrição;
- `Disease_Treatment.csv` que contem uma lista de doenças e a cada uma associa uma lista de tratamentos;
- `doentes.json` que a um nome de doente associa uma lista de sintomas.

Vais usar estes datasets para povoar a ontologia da seguinte forma:

1. A partir de `Disease_Syntoms.csv` vais criar instâncias de doença (`:Disease`) para cada doença;
2. Para os sintomas de cada doença vais criar uma instância para cada sintoma se esta ainda não existir;
3. Vais associar cada doença criada aos respetivos sintomas;
4. A partir de `Disease_Description.csv` vais associar uma descrição a cada doença (cria esta nova propriedade);
5. Depois destas alterações, grava a ontologia com o nome `med_doencas.ttl`;
6. A partir de `Disease_Treatment.csv` vais criar uma instância para cada tratamento se esta ainda não existir;
7. Vais associar cada doença criada aos respetivos tratamentos;
8. Depois destas alterações, grava a ontologia com o nome `med_tratamentos.ttl` (é sempre o resultado cumulativo de todas as alterações feitas até ao momento);
9. A partir de `doentes.json` vais criar as instâncias dos doentes: um doente deverá ter um id (usa a tua imaginação), um nome (extraído do dataset) e uma lista de sintomas associados (extraídos do dataset);

10. Depois destas alterações, grava a ontologia com o nome `med_doentes.ttl`;

Interrogando...

- Guardas as queries SPARQL que desenvolveres num ficheiro de nome `sparql.txt`, identificando claramente a questão a que a respetiva query responde.

11. Uma vez criada a ontologia (última versão: `med_doentes.ttl`), especifica queries SPARQL que permitam responder às seguintes questões:
    - Quantas doenças estão presentes na ontologia?
    - Que doenças estão associadas ao sintoma `"yellowish_skin"`?
    - Que doenças estão associadas ao tratamento `"exercise"`?
    - Produz uma lista ordenada alfabeticamente com o nome dos doentes.
12. Cria uma query CONSTRUCT que diagnostique a doença de cada pessoa, ou seja, produza uma lista de triplos com a forma `:patientX :hasDisease :diseaseY`. No fim, acrescenta estes triplos à ontologia;
13. Cria um query SPARQL que poduza uma distribuição dos doentes pelas doenças, ou seja, dá como resultado uma lista de pares `(doença, nº de doentes)`;
14. Cria um query SPARQL que poduza uma distribuição das doenças pelos sintomas, ou seja, dá como resultado uma lista de pares `(sintoma, nº de doenças com o sintoma)`;
15. Cria um query SPARQL que poduza uma distribuição das doenças pelos tratamentos, ou seja, dá como resultado uma lista de pares `(tratamento, nº de doenças com o tratamento)`;
16. Cria um query SPARQL que poduza uma lista das doenças e respetivos sintomas, ou seja, dá como resultado uma lista de pares `(doença, lista de sintomas)`.

Bom trabalho e boa sorte jcr